

Name: ycchiu226

Date: 2026-09-29

Topic: Testing, Debugging, Exceptions, Assertions

1. Today's Challenge

Core concept from today's OCW lecture:

Testing, debugging, and avoiding errors when modifying a collection during iteration.

AI-generated coding challenge title:

Remove Target Values In-Place

2. My Initial Approach — Before AI Help

My approach:

I planned to traverse the list in order, identify elements matching the target, and remove them. When an element was removed, the following elements would shift one position forward. I intended to use an index-controlled loop and handle the index carefully to avoid skipping elements.

3. AI Tutor Help

Did you ask the AI Tutor for help?

☐ No — I solved it independently

☒ Yes — I received one or more hints

The most useful hint/question from AI was:

After removing an element, what happens to the next element, and could it be skipped if the index advances?

It helped me realize that:

Removing an element shifts the following elements to the left. Therefore, I should keep the index unchanged after a removal so that the element shifted into the current position is checked next.

4. My Revision

Did you change your approach or code after interacting with AI?

☐ No

☒ Yes

What did you change, and why?

I chose an index-controlled `while` loop. I increment the index only when the current element is not the target. If the element is removed, I leave the index unchanged so the next shifted element is not skipped.

5. Verification

My final program:

- ☒ Passed the provided examples
- ☒ Passed additional edge cases
- ☐ Still has unresolved problems

One edge case I tested:

Input: `L = [6, 6, 6, 6], target = 6`

Expected output: `[]`

Actual output: `[]`

6. One-Minute Reflection

What idea from the OCW lecture did you transfer to this new problem?

I applied in-place mutation, careful boundary handling, and the importance of avoiding errors when modifying a collection during iteration.

One thing I understand better now:

I understand how list mutation affects index positions during traversal. By keeping the index unchanged after removing an element, I can avoid skipping consecutive target values.

One thing I am still unsure about:

I am still unsure how to improve the time complexity of this in-place removal approach while preserving the original order of the remaining elements.